DA3408 - AI-OPS Lab

Assignment 1 — Experiment Management & Reproducibility

Vangala VedaVachan Reddy DA24B029

August 2026

Question 1

1.1

(a) **Entanglement (CACE – "Changing Anything Changes Everything")**

Entanglement means that a change in one part of an ML system can unexpectedly affect another part. For example- if we change how the system rounds the estimated delivery time it may also change the favorite restaurant recommendations even though they are not directly related so this shows that different parts of the system are connected in ways we may not notice.

(b) **Undeclared Consumers**

The marketing dashboard is using the model's raw output without the ML team knowing about it. This is called Undeclared Consumers because another team or system is depending on the model's output without clearly telling or documenting it and this can cause problems if the ML team changes the model or its output later.

(c) **Configuration & Glue-Code Debt**

The training process uses 14 shell scripts that are not properly documented or organized and there is no tool managing how they run so this is called Configuration & Glue-Code Debt because the system depends on many small scripts and settings that are scattered around over time this makes the pipeline hard to understand, maintain and fix.

1.2 Mitigation for (c)

A good solution is to use Git and MLflow to keep the training process organized. Git can store and track changes in the code and settings while MLflow can keep track of things like the learning rate, batch size, model structure, training loss, and accuracy.

This makes it easier to see what changed, compare experiments, and find mistakes and instead of depending on 14 undocumented shell scripts, the whole training process becomes clear, organized and easy to track.

Question 2

Experimental Results

Six experiments were performed using a Multi-Layer Perceptron (MLP) model on the MNIST dataset by varying the learning rate and batch size. Table 1

Table 1: MLP Experiment Results

Experiment	Learning Rate	Batch Size	Train Loss	Validation Accuracy
MLP-lr0.001-batch32	0.001	32	0.049410	0.9717
MLP-lr0.001-batch64	0.001	64	0.048975	**0.9762**
MLP-lr0.005-batch32	0.005	32	0.113562	0.9673
MLP-lr0.005-batch64	0.005	64	0.090925	0.9714
MLP-lr0.01-batch32	0.010	32	0.193253	0.9490
MLP-lr0.01-batch64	0.010	64	0.147632	0.9614

Analysis

This results shows the best-performing configuration was **MLP-lr0.001-batch64** which achieved the highest validation accuracy of **97.62%** and the lowest training loss of **0.048975** and it indicates that the combination of a learning rate of 0.001 and a batch size of 64 provided the most effective and the learning rate had a noticeable effect on model performance and the average validation accuracy for learning rates 0.001, 0.005, and 0.01 was **97.395%**, **96.935%**, and **95.520%**, respectively.the average validation accuracy for batch sizes 32 and 64 was **96.267%** and **96.967%**, respectively. Thus, batch size 64 performed slightly better on average than batch size 32. overall the experiments indicate that **learning rate had a larger influence on validation performance than batch size** and the learning rate of 0.001 consistently produced the highest validation. therefore among the tested configurations, **learning rate = 0.001 and batch size = 64** was the best-performing configuration, achieving a validation accuracy of **97.62%**.